Manual técnico de NovaMut

Oct 20, 2026 · @Fox

Resumen

NovaMut 4.0 es una aplicación de escritorio para gestión de mutuales, escrita en Python como **un solo archivo** (`novamut.py` , 5.836 líneas) con una base **SQLite** local (`novamut.db`). La interfaz es una ventana única con menú lateral y paneles intercambiables, hecha con **CustomTkinter**.

Componente	Uso	Obligatorio
Python 3.9+ (probado en 3.12 a 3.14)	Intérprete	Sí
customtkinter + pillow	Interfaz gráfica	Sí: sin él el programa termina con un mensaje
sqlite3, csv, hashlib (biblioteca estándar)	Datos, CSV, contraseñas	Incluidos en Python
reportlab	PDF (credenciales y reportes)	No: se importa recién al generar un PDF
pyinstaller	Compilar el .exe	Solo para distribuir

Carpeta de trabajo:

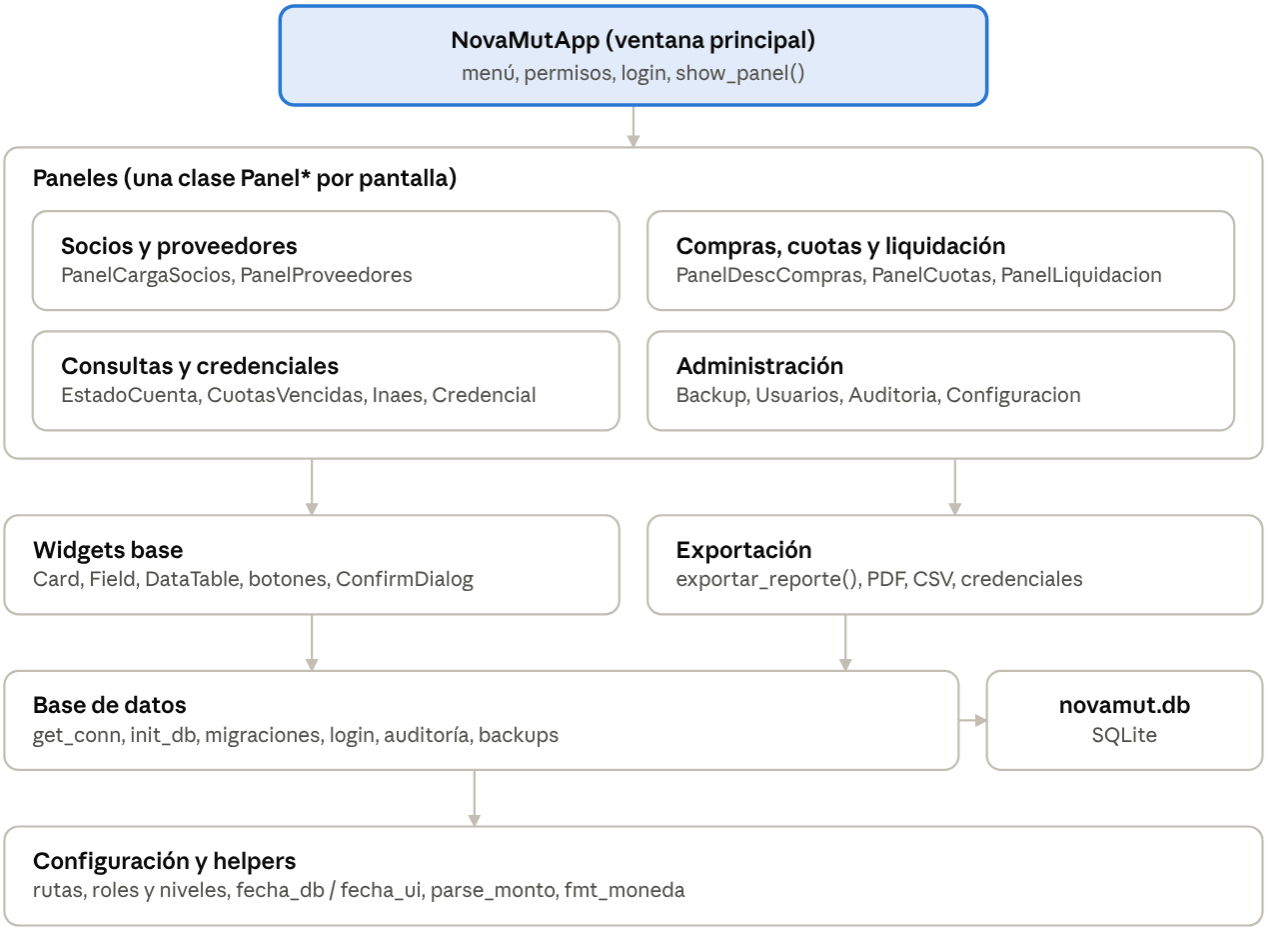
```
NovaMut
 ├─ novamut.py     código
 ├─ novamut.db     base SQLite (se crea si no existe)
 ├─ backups\       backups automáticos (se crea al cerrar)
 ├─ INAES\         padrones emitidos (opcional)
 └─ .venv\         entorno virtual
```

Autores: Diego Alejandro Fernandez y Joshua Giuliano Venticinque. Uso y distribución sujetos a los términos incluidos en el programa.

Estructura del código

El archivo está ordenado en capas, de arriba hacia abajo: configuración, funciones auxiliares, base de datos, componentes visuales, la ventana principal y después un panel por pantalla. Cada sección empieza con un comentario `# ====` y su título.

La ventana muestra un panel a la vez; cada capa usa la de abajo



capas de novamut.py · de la ventana a la base

La ventana principal crea los paneles; los paneles usan los widgets, la exportación y la base de datos, y todo se apoya en la configuración y los helpers del principio del archivo.

Líneas	Sección	Contenido principal
1–91	Configuración visual	Imports, paleta `COLOR_*`, fuente `F()`, `MESES`
92–121	Rutas y persistencia	`_base_dir()`, `_recurso()`, `APP_VERSION`, `DB_PATH`, `BACKUP_DIR`, `SCHEMA_VERSION`
122–172	Roles y permisos	`ROLES`, `ROL_NIVEL`, `NIVEL_OPERAR`, `NIVEL_ADMIN`, `tiene_nivel()`, numeración de socios

Líneas	Sección	Contenido principal
173–272	Validadores y helpers	`parse_fecha()`, `fecha_db()`, `fecha_ui()`, `parse_monto()`, `fmt_moneda()`, `sumar_meses()`
273–653	Base de datos	`get_conn()`, contraseñas, migraciones, `init_db()`, `verificar_login()`, config, auditoría, backups
654–914	Widgets base	`Card`, `Toast`, `Field`, botones, `Badge`, `DataTable`, `ConfirmDialog`
915–1287	App principal	`NovaMutApp`: menú, permisos, navegación, flujo de login
1288–1378	Login	`PanelLogin`
1379–2296	Socios y proveedores	`PanelCargaSocios`, `VentanaListaSocios`, `PanelProveedores`, `VentanaListaProveedores`
2297–3045	Compras	`plan_cuotas()`, `control_credito()`, `PanelDescCompras`
3046–3306	Cuotas	`PanelCuotas`
3307–3473	Exportación	`exportar_csv()`, `exportar_pdf()`, `exportar_reporte()`
3474–3747	Liquidación y configuración	`PanelLiquidacion`, `PanelConfiguracion`
3748–4509	Consultas	`PanelEstadoCuenta`, `PanelCuotasVencidas`, `PanelInaesRes756` (y `PanelConsultas`, sin entrada en el menú)
4510–5300	Administración	`PanelBackup`, `PanelCambiarPassword`, `PanelGestionUsuarios`, `PanelAuditoria`
5301–5699	Credenciales	`dibujar_credencial()`, PDF individual y A4, `PanelCredencialSocio`
5700–5832	Términos y programadores	`TERMINOS_VERSION`, `TERMINOS_TXT`, `PanelTerminos`, `PanelProgramadores`
5833–fin	Arranque	`init_db()` y `NovaMutApp().mainloop()`

Cada pantalla es una clase `Panel*` que hereda de `ctk.CTkFrame` y recibe `(master, app, titulo, **kwargs)`. `app.show_panel(clase, **kwargs)` destruye el panel anterior y crea el nuevo; el panel accede al usuario con `self.app.usuario` y avisa con

`self.app.toast(msg, tipo)`.

Modelo de datos

La base tiene 10 tablas. El núcleo es **una compra** (`desc_compras_sno`) de un socio a un proveedor, que genera dos planes de cuotas: lo que se le cobra al socio (`cuotas_socio`) y lo que se le paga al proveedor (`cuotas_proveedor`).

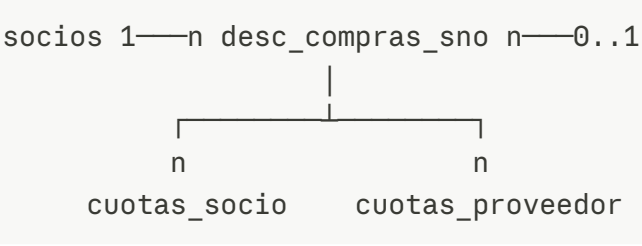


Tabla	Clave	Columnas importantes
`meta`	`clave`	`schema_version`; configuración con prefijo `cfg_` (por ejemplo `cfg_entidad_nombre`)
`usuarios`	`id` (autoinc.)	`usuario` (único, minúsculas), `password` (hash), `salt`, `rol`, `activo`, `debe_cambiar_pwd`, `intentos_fallidos`, `bloqueado_hasta`, `terminos_version`, `terminos_fecha`
`log_accesos`	`id`	`usuario`, `fecha`, `exito`, `host`
`log_auditoria`	`id`	`fecha`, `usuario_actor`, `accion`, `detalle`
`socios`	`nro_socio` (asignado por el sistema, desde 1001)	datos personales, `cuil`, `fec_nac`, `fec_ing`, `fec_baj`, `fec_egr`, `credit_max`, `cre_max_mes`, `tope_emp`, `cbu`, `usr_alta` / `fec_alta`, `usr_mod` / `fec_mod`
`proveedores`	`cod_proveedor` (autoinc.)	`razon_social`, `cuit`, `condicion_iva`, `cbu`, `activo`

Tabla	Clave	Columnas importantes
`desc_compras_sno`	`id`; único (`tip_compro`, `nro_compro`)	`nro_socio` (FK), `cod_proveedor` (FK, puede ser NULL), `importe`, `can_cuot_so`, `can_cuot_pr`, `fec_factura`, `fe_1cu_soc`, `fe_1cu_pro`, `estado` (ACTIVO / ANULADO)
`cuotas_socio`	`id`	`desc_compra_id` (FK, cascade), `nro_cuota`, `fec_vencimiento`, `importe`, `fec_cobro`, `estado` (PENDIENTE / COBRADA / ANULADA)
`cuotas_proveedor`	`id`	igual, con `fec_pago` y estado PAGADA

Nada se borra físicamente: socios y proveedores se dan de baja (fecha de baja o `activo = 0`), las compras se anulan y las cuotas cambian de estado. `get_conn()` activa `PRAGMA foreign_keys = ON` en cada conexión.

Convenciones

Fechas. En la base van como texto ISO `aaaa-mm-dd`, así ordenan bien con `ORDER BY` y se agrupan por mes con `substr(fecha, 1, 7)`. En pantalla se muestran y se escriben como `dd/mm/aaaa`. Siempre convertir en el borde:

Función	Uso
`fecha_db(s)`	pantalla → base. Vacío devuelve `""`, inválida devuelve `None`
`fecha_ui(s)`	base → pantalla. Acepta también el formato viejo
`parse_fecha(s)`	texto → `datetime`, acepta `dd/mm/aaaa`, `dd-mm-aaaa` y `aaaa-mm-dd`
`hoy_db()` / `hoy_ui()`	fecha de hoy en cada formato

Las fechas de auditoría (`fec_alta`, `fec_mod`, `log_*.fecha`) son `datetime.isoformat()` con hora.

Importes. Se guardan como `REAL` redondeados a 2 decimales con `parse_monto()` (usa `Decimal`, acepta coma o punto). Se muestran con `fmt_moneda()` en formato argentino (`$1.234,56`) y se exportan a CSV con `fmt_numero_csv()` (`1234,56`).

CUIL/DNI. Se guardan como se escriben. Para comparar se usan solo los dígitos (`solo_digitos()` en Python, `REPLACE` de `-`, espacio y `.` en SQL).

Rutas. `_base_dir()` es la carpeta del .exe (si está compilado) o la de `novamut.py`. Ahí viven `novamut.db`, `backups\` e `INAES\`. `_recurso(nombre)` busca archivos empaquetados con PyInstaller (`sys._MEIPASS`), por ejemplo el ícono.

Textos. La interfaz usa español sin tildes en la mayoría de los textos fijos, para evitar problemas de codificación en Windows. Los mensajes al usuario van por `app.toast(msg, tipo)` (`info`, `success`, `warning`, `error`) y las confirmaciones por `ConfirmDialog.yes_no()`.

Migraciones y base anterior

`init_db()` corre en cada arranque y es idempotente:

1. `_adoptar_base_vieja()`: si no existe `novamut.db` pero sí `sigemut.db` (nombre anterior), la copia con la API de backup de SQLite. La vieja queda intacta.
2. `CREATE TABLE IF NOT EXISTS` de todas las tablas.
3. `_aplicar_migraciones()`: lee `meta.schema_version` y aplica los pasos de `_MIGRACIONES` desde esa versión hasta `SCHEMA_VERSION`. Los `ALTER TABLE` que fallan porque la columna ya existe se ignoran.
4. Si no queda ningún SUPERADMIN activo, promueve al usuario más antiguo.
5. Si no hay usuarios, crea `admin` / `admin` con cambio de clave obligatorio.

Versión	Cambios
2	`usuarios`: `debe_cambiar_pwd`, `intentos_fallidos`, `bloqueado_hasta`, `creado_por`. Tabla `log_auditoria`
3 (actual)	`usuarios`: `terminos_version`, `terminos_fecha`. `_migrar_fechas_iso()` convierte las fechas de `socios`, `desc_compras_sno`, `cuotas_socio` y `cuotas_proveedor` de `dd/mm/aaaa` a `aaaa-mm-dd`

Las columnas convertidas están en `_COLUMNAS_FECHA`. La conversión saltea los valores que ya están en ISO, así que se puede repetir sin daño.

Seguridad

Contraseñas. PBKDF2-SHA256 con 260.000 iteraciones y salt aleatorio de 16 bytes por usuario, guardado como `pbkdf2$<iter>$<hex>`. `verify_password()` también acepta el formato viejo (SHA-256 plano) y `verificar_login()` lo re-hashea a PBKDF2 en el primer ingreso exitoso. Política: mínimo 8 caracteres con letras y números (`val_password_fuerte()`).

Bloqueo. `MAX_INTENTOS_LOGIN = 5` fallos seguidos bloquean la cuenta `MINUTOS_BLOQUEO = 15` minutos (`bloqueado_hasta`). Cada intento, exitoso o no, se guarda en `log_accesos` con el nombre de la PC.

Roles. Niveles en `ROL_NIVEL`: CONSULTA 1, OPERADOR 2, ADMIN 3, SUPERADMIN 4. Se controlan en tres lugares:

Dónde	Cómo
Menú	`MENU_ITEMS` tiene un permiso por ítem (`None`, `"operar"`, `"admin"`). `_set_sidebar_visible()` oculta lo que no corresponde y `_on_sidebar_click()` lo vuelve a verificar
Botones	`deshabilitar_sin_nivel(app, botones, nivel)` los deshabilita al crear el panel
Acciones	Cada método que modifica datos empieza con `if not self.app.puede(NIVEL_OPERAR, "..."): return`. Es el control que realmente impide la acción; los dos anteriores son visuales

Acciones por nivel: `NIVEL_OPERAR` para altas, modificaciones, bajas, anulaciones, cobros, liquidación, padrón y backups; `NIVEL_ADMIN` para revertir cobros y configuración. La gestión de usuarios usa `puede_gestionar_rol()` (nadie gestiona a un par ni a un superior, salvo SUPERADMIN) y `roles_asignables()`.

Términos. `on_login_ok()` compara `usuarios.terminos_version` con `TERMINOS_VERSION`. Si no coinciden, muestra `PanelTerminos`. Aceptar guarda la versión y la fecha y audita `ACEPTA_TERMINOS`; rechazar audita `RECHAZA_TERMINOS` y cierra el programa. Para obligar a todos a aceptar de nuevo, subir `TERMINOS_VERSION`.

Flujo de ingreso: `PanelLogin` → `verificar_login()` → `on_login_ok()` → (términos si hace falta) → `_entrar()` → (cambio de clave si `debe_cambiar_pwd`) → Socios.

Alcance. La base no está cifrada: quien tenga acceso al archivo `novamut.db` puede leerlo con cualquier visor de SQLite. Los permisos protegen el uso del programa, no el archivo.

Lógica de negocio

Numeración de socios

`proximo_nro_socio(con)` devuelve `max(NRO_SOCIO_INICIAL, MAX(nro_socio) + 1)`, con `NRO_SOCIO_INICIAL = 1001`. En el alta, el número se recalcula dentro de un `BEGIN IMMEDIATE` justo antes del `INSERT`, así dos PCs no pueden tomar el mismo. El campo está siempre deshabilitado en la ficha.

Compras y cuotas

`plan_cuotas(importe, cantidad, primera)` devuelve `[(fecha, importe)]`: cuotas iguales redondeadas a 2 decimales, la diferencia en la última, y vencimientos mensuales con `sumar_meses()` (respeta fin de mes). `_generar_cuotas()` lo usa para `cuotas_socio` y, solo si hay proveedor, para `cuotas_proveedor`. La primera cuota usa `fe_1cu_*` o, si falta, `fec_factura`; sin ninguna fecha no se deja guardar.

Al **modificar**:

- Si cambian importe, cantidades, fechas, socio o proveedor (`cambia_plan`) y no hay cuotas canceladas, se borran y se regeneran todas.
- Si hay cuotas PAGADAS o COBRADAS (`cuotas_ya_canceladas()`), ese cambio se rechaza. Solo pasan concepto, factura y observaciones.

Anular pone la compra en ANULADO y sus cuotas pendientes en ANULADA. **Revertir** un cobro de una compra anulada deja la cuota en ANULADA, no en PENDIENTE.

Control de crédito

`control_credito(con, nro_socio, importe, plan, excluir_desc_id)` devuelve una lista de avisos (vacía si está todo bien o el socio no tiene topes):

- `credit_max > 0`: `deuda_pendiente_socio()` + importe no debe superarlo. La deuda es la suma de cuotas PENDIENTES de compras ACTIVAS.
- `cre_max_mes > 0`: para cada mes del plan suma lo ya pendiente en ese mes (`substr(fec_vencimiento,1,7)`) más la cuota nueva, e informa el peor mes.

`excluir_desc_id` evita contar la propia compra al modificarla. Si hay avisos, se pide confirmación y, si se acepta, se audita `CREDITO_EXCEDIDO`.

Liquidación

`PanelLiquidacion._refrescar()` toma las `cuotas_socio` PENDIENTES de compras ACTIVAS con `substr(fec_vencimiento,1,7) = 'aaaa-mm'` (o `<=` si se incluyen atrasadas), las agrupa por socio en `self.resumen` y guarda el detalle en `self.filas`.

`_cobrar()` hace un `executemany` sobre esos `id` con `WHERE estado='PENDIENTE'`, para no tocar cuotas que otro usuario haya cambiado mientras tanto, y audita `COBRO_LIQUIDACION`.

Reportes, PDF y credenciales

`reportlab` se importa recién cuando se genera un PDF (`_importar_reportlab()` avisa si falta), así el programa abre sin él.

`exportar_reporte(app, titulo, subtitulo, columnas, filas, totales, nombre_archivo, accion_auditoria)` es el punto único de exportación: abre el diálogo de guardar y, según la extensión elegida, llama a:

- `exportar_csv()`: separador `;`, codificación `utf-8-sig` (BOM) para que Excel en español lo abra bien.
- `exportar_pdf()`: `SimpleDocTemplate` de platypus, A4 vertical (o apaisado con más de 6 columnas), encabezado con el nombre de la entidad, tabla con encabezado repetido por página, totales y pie con versión y número de página.

Las columnas se definen como `(titulo, clave, ancho_relativo, tipo)`, con `tipo` en `txt`, `fecha`, `moneda` o `int`; `_fmt_celda()` formatea cada valor según el destino. Para agregar un reporte nuevo alcanza con armar `filas` (lista de dicts) y llamar a `exportar_reporte()`.

Credenciales. `dibujar_credencial(pdf, socio, x0, y0)` dibuja una tarjeta de 8,6 × 5,4 cm (`CRED_ANCHO_CM`, `CRED_ALTO_CM`) en cualquier posición. `generar_pdf_credencial()` hace una por página del tamaño de la tarjeta, y `generar_pdf_credenciales_a4()` ubica 2 × 5 por hoja A4 con marcas de corte. Los nombres largos se achican hasta 7 pt y después se parten en 2 líneas con `simpleSplit`.

Padrón INAES. `PanelInaesRes756._emitir()` escribe CSV con `;`, fechas `AAAAMMDD` (`fmt_inaes()`) y la columna `TIPO_MOVIMIENTO`; divide en varios archivos según "registros por archivo" y deja los faltantes en `*_ERRORES.LOG`.

Auditoría y backups

`registrar_auditoria(usuario, accion, detalle)` agrega una fila a `log_auditoria`. En las modificaciones, `describir_cambios(antes, despues)` arma el detalle con los campos que cambiaron (`campo: viejo -> nuevo`, hasta 8).

Grupo	Acciones registradas
Socios y proveedores	`ALTA_SOCIO`, `MODIFICA_SOCIO`, `BAJA_SOCIO`, `ALTA_PROVEEDOR`, `MODIFICA_PROVEEDOR`, `BAJA_PROVEEDOR`, `ACTIVA_PROVEEDOR`
Compras y cuotas	`ALTA_COMPRA`, `MODIFICA_COMPRA`, `ANULA_COMPRA`, `CREDITO_EXCEDIDO`, `PAGO_CUOTA`, `COBRO_CUOTA`, `REVERTIR_PAGO`, `REVERTIR_COBRO`, `COBRO_LIQUIDACION`
Reportes	`EXPORTA_REPORTE`, `EXPORTA_LIQUIDACION`, `CREDENCIAL_PDF`, `CREDENCIALES_PDF`, `EMITIR_PADRON_INAES`
Sistema	`BACKUP`, `CONFIGURACION`, `ACEPTA_TERMINOS`, `RECHAZA_TERMINOS`, altas y cambios de usuarios, cambios y reseteos de clave

Backups. `hacer_backup(destino, origen)` usa `sqlite3.Connection.backup()`, que es seguro con la base en uso (no se copia el archivo a mano). `backup_automatico()` se llama desde `NovaMutApp._al_cerrar()` (el evento `WM_DELETE_WINDOW`): guarda `backups/novamut_auto_AAAAMMDD_HHMMSS.db`, conserva las últimas `BACKUPS_AUTO_A_CONSERVAR = 30` y nunca lanza excepciones, para no impedir el cierre. Si el programa se corta de golpe (corte de luz, cerrar el proceso), ese backup no se hace.

Entorno, ejecución y compilación

Preparar el entorno (Windows)

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install customtkinter pillow reportlab
pyinstaller
```

Si `.venv` se copió desde otra carpeta y deja de funcionar, hay que borrarlo y crearlo de nuevo: guarda rutas absolutas.

Ejecutar

```
.venv\Scripts\python.exe novamut.py
```

En VS Code: `Ctrl+Shift+P` → **Python: Select Interpreter** → `.venv`. Para probar con datos sin tocar la base real, copiar `novamut.py` y un backup (renombrado `novamut.db`) a otra carpeta: el programa siempre usa la base de su propia carpeta.

Compilar el .exe

```
.venv\Scripts\python.exe -m PyInstaller --noconfirm --onefile --windowed --
name NovaMut --collect-all customtkinter --collect-all reportlab novamut.py
```

- `--collect-all customtkinter` es obligatorio: sin los temas JSON de customtkinter el .exe se cierra al abrir.
- Con ícono: agregar `--icon novamut.ico --add-data "novamut.ico;."`. `_recurso()` lo encuentra dentro del .exe.
- Resultado: `dist\NovaMut.exe`. Se distribuye junto a `novamut.db` o solo (crea una base nueva con `admin` / `admin`).
- `build\` y `NovaMut.spec` se pueden borrar.

Antes de entregar, conviene probar el .exe en una PC sin Python. Algunos antivirus marcan como sospechosos los .exe de PyInstaller (falso positivo).

Cómo extender

Agregar una pantalla

1. Crear la clase, por ejemplo antes de `# PANEL: CONSULTAS`:

```
class PanelNuevo(ctk.CTkFrame):
    def __init__(self, master, app, titulo="Nuevo", **kwargs):
        super().__init__(master, fg_color="transparent")
        self.app = app
        app.set_breadcrumb("Seccion", "Titulo de la pantalla")
        app.set_badge("")
        card = Card(self, title="Datos")
        card.pack(fill="x", pady=6)
```

2. Agregarla a `NovaMutApp.MENU_ITEMS`: `("Nuevo", "◎", "nuevo", None)`. El último valor es el permiso para verla: `None`, `"operar"` o `"admin"`.
3. Agregarla al diccionario `panels` de `_on_sidebar_click()`: `"nuevo": (PanelNuevo, "Nuevo")`.

Proteger una acción

```
def _guardar(self):
    if not self.app.puede(NIVEL_OPERAR, "guardar esto"):
        return
    ...
    registrar_auditoria(self.app.usuario["usuario"], "MI_ACCION", detalle)
```

Y, para que se vea deshabilitado, `deshabilitar_sin_nivel(self.app, [boton])` al crear el panel.

Agregar un cambio de esquema

1. Subir `SCHEMA_VERSION` (por ejemplo, a 4).
2. Agregar en `_MIGRACIONES` la clave `4` con la lista de SQL: `4: ["ALTER TABLE socios ADD COLUMN legajo TEXT"]`.
3. Si hace falta transformar datos con Python, agregar un `if v == 4:` en `_aplicar_migraciones()`, como hace la versión 3 con las fechas.
4. Agregar también la columna en el `CREATE TABLE` de `init_db()`, para las bases nuevas.

Las bases existentes se actualizan solas la próxima vez que se abre el programa. Probar siempre la migración sobre una copia de una base real.

Otros puntos de ajuste

Qué	Dónde
Primer número de socio	`NRO_SOCIO_INICIAL`
Intentos y minutos de bloqueo	`MAX_INTENTOS_LOGIN`, `MINUTOS_BLOQUEO`
Backups automáticos a conservar	`BACKUPS_AUTO_A_CONSERVAR`
Texto de los términos	`TERMINOS_TXT` (y subir `TERMINOS_VERSION`)
Programadores y teléfonos	`PROGRAMADORES`
Colores	constantes `COLOR_*`
Versión	`APP_VERSION`

Limitaciones conocidas

Tema	Situación actual	Posible mejora
Varias PCs	SQLite en una carpeta compartida funciona con 2 o 3 PCs en red local, pero puede trabarse o dañarse con conexiones inestables	Servidor de base de datos (PostgreSQL) o servicio central
Base sin cifrar	Cualquiera con acceso a `novamut.db` puede leerla	Permisos de carpeta en Windows; cifrado (SQLCipher)
Importes	Se guardan como `REAL` (coma flotante) redondeado a 2 decimales	Guardar centavos como `INTEGER`
Pagos parciales	Una cuota se cobra o paga entera	Tabla de pagos con saldo por cuota
Pagos a proveedores	Solo de a una cuota; no hay liquidación por proveedor	Pantalla equivalente a Liquidación para egresos
Deshacer una liquidación	Se revierte cuota por cuota (solo ADMIN)	Revertir por lote, usando la fecha y la auditoría
Backup automático	Solo al cerrar normalmente; un corte de luz lo saltea	Backup también al iniciar o cada cierto tiempo
Listados	Topes: 500 socios en la lista, 1000 cuotas, 500 registros de auditoría	Paginación
Cierres	La etiqueta "Último cierre" de Desc/Compras no tiene función	Implementar cierres mensuales que bloqueen cambios
Código sin uso	`PanelConsultas` quedó sin entrada en el menú	Borrarla o volver a usarla
Pruebas	No hay pruebas automatizadas	Pruebas con `pytest` sobre las funciones de negocio (`plan_cuotas`, `control_credito`, migraciones)
Tamaño del archivo	5.836 líneas en un solo archivo	Dividir en módulos (ya probado: funciona con importación en cadena)